COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S1
DATE PERFORMED:	10/13/26	**DATE SUBMITTED:**	
NAME:	Reyes, Eli Justin L. Santos, Adrian Walter D.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 12.1 **[Hands-on Activity 12.1 Algorithmic Strategies]**

1. PROCEDURE OUTPUT

Strategy	Algorithm Analysis	Analysis
Recursion	Pre order Tree Traversal	Pre-order traversal processes a tree node by visiting the root first, then recursively traversing the left subtree and right subtree. It relies on the recursive call stack to break the problem down into smaller sub-tree operations until base cases (leaf nodes) are reached.
Brute Force	Linear Search	Linear search is the simplest form of searching as it works by looking through every element in the array. It is an expensive searching algorithm as its total operations are directly dependent on the number of elements that the array has. All this follows the definition of a brute force algorithm strategy.
Backtracking	In order Tree traversals	Since the tree traversals work by going back to the root. It fulfils backtracking which goes back to the first element.
Greedy	Selection sort	A greedy algorithm works by finding the best possible choice and ignoring all others. This makes it an efficient algorithm to work with
Divide-and-Conquer	Merge Sort	Merge sort uses a divide and conquer algorithm in sorting the array. It divides the array into sub arrays until it is sorted then merges

		every element.

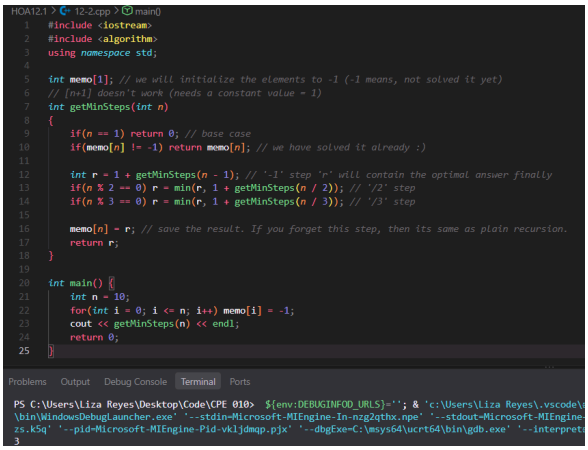
Screenshot	
Analysis	Top-down approach has minimum steps starting from n down to 1 by breaking the problem into recursive calls for n - 1, n / 2, and n / 3. It checks a global memo array initialized to -1 so previously solved inputs return cached results instead of recomputing. (We set it to 100 cause it wouldn't work with memo[n+1] it needed a value inside it)

Table 12-2. Memoization Implementation

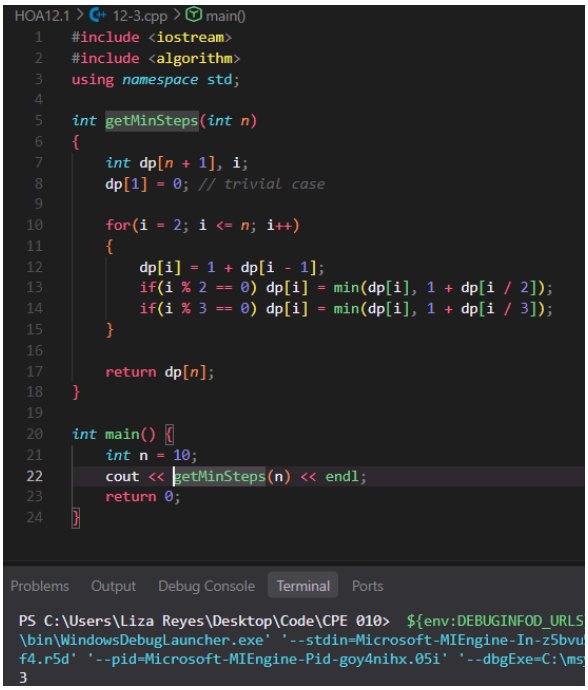
Screenshot	
Analysis	Bottom-up eliminates recursion by repeatedly solving subproblems from dp[1] = 0 up to n using a single loop. Each step evaluates dp[i - 1], dp[i / 2], and dp[i / 3] using already stored array values to build the answer.

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Pseudo code	FUNCTION pathFind(matrix, row, col, remain) IF row >= matrix.rows OR col >= matrix.cols RETURN 0 remain = remain - matrix[row][col] IF remain < 0 RETURN 0 IF row == matrix.rows - 1 AND col == matrix.cols - 1 IF remain == 0 RETURN 1 ELSE RETURN 0

```
    right = pathFind(matrix, row, col + 1, remain)
    down  = pathFind(matrix, row + 1, col, remain)

    RETURN right + down
END FUNCTION

START
    Store matrix
    Print matrix
    Input cost
    Print pathFind(matrix, 0, 0, cost)
END
```

Code

```cpp
#include <iostream>

using namespace std;

const int GRID_ROWS = 4;
const int GRID_COLS = 4;

int grid[GRID_ROWS][GRID_COLS] = {
    // Lab supple given matrix
    {4, 7, 1, 6},
    {6, 7, 3, 9},
    {3, 8, 1, 2},
    {7, 1, 7, 3}
};

int countValidPaths(int currentRow, int currentCol, int targetCost) {
    if (currentRow >= GRID_ROWS || currentCol >= GRID_COLS) {
        return 0;
    }

    targetCost = targetCost - grid[currentRow][currentCol];
    if (targetCost < 0) {
        return 0;
    }

    if (currentRow == GRID_ROWS - 1 && currentCol == GRID_COLS - 1) {
        if (targetCost == 0) {
            return 1;
        }
        return 0;
    }

    int rightWays = countValidPaths(currentRow, currentCol + 1, targetCost);
    int downWays = countValidPaths(currentRow + 1, currentCol, targetCost);
    return rightWays + downWays;
}
```

```cpp
int main() {
    int userBudget;
    cout << "Enter required cost: ";
    cin >> userBudget;

    int totalPaths = countValidPaths(0, 0, userBudget);
    cout << "Number of paths with cost " << userBudget << ": " << totalPaths << endl;

    return 0;
}
```

Output

```
Enter required cost: 34
Number of paths with cost 34: 1
```

Code analysis	
Reyes	Following the pseudocode, the algorithm only moves right or down and operates on a simple flow. It returns 0 if a path goes off the grid or if the remaining cost drops below zero. It returns 1 only when reaching the destination with a remaining cost of exactly zero. Finally, it adds the total successful paths returned from both the right and down recursive calls.
Santos	The code uses a backtracking strategy to recursively find all valid paths through the matrix. It tests moving right and down step-by-step while subtracting each cell's cost from the remaining total. If a route goes out of bounds or exceeds the allowed budget, it stops immediately and backtracks to try another direction.

Conclusion	
Reyes	This activity covered key strategies like recursion, backtracking, brute force, greedy, and divide-and-conquer. We identified algorithms matching each strategy, explored dynamic programming (memoization and tabulation), and applied recursive backtracking on a 4x4 matrix to find a target path cost.
Santos	This activity reinforced key lecture topics on algorithm design, including recursion, brute force, backtracking, greedy methods, and divide-and-conquer. The supplementary exercise applied recursive traversal on a matrix to find paths matching a specific cost.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance

with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.